

function detects all blobs and their co-ordinates are stored in a matrix called blobs.

Display all images. The following function shows all the processed images side by side (Fig. 12):

```
result = img.sideBySide(imgdist,
side='bottom', scale=False)
result = result.sideBySide(imgdistbin,
side='right')
result.show()
```

Count blobs. The number of blobs is equal to the number of tablets. Below function counts the number of blobs:

```
if blobs is not None:
    le = len(blobs)
    print 'Pills Detected =', le
```

The first statement helps the code to keep running even if no blob is detected. Otherwise, the program will terminate in the next step. The second statement counts the length of the matrix, which is equal to the number of blobs. The third statement prints the number of blobs detected.

Check and compare the blobs detected. The code for this purpose follows:

```
if le == 3:
    winsound.Beep(1000, 100)
    print 'OK'
else:
    print 'FAIL'
```

The first statement compares the number of detected blobs with the prefixed number of pills in the pack. Here, the fixed value is entered as 3 because three-tablet blister packs were used for testing purpose. You need to change the number as required. The second statement plays a beep (frequency, time) if the number of blobs detected is 3 and the third statement prints OK. Otherwise, correct blob detection fails.

How the system works

The blister-packed medical pills are manufactured through automated processes. These processes are generally carried out using a conveyor system. The packs are put

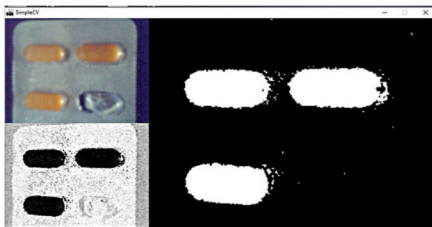


Fig. 12: Display of all images

on the conveyor belt, inspected and packed. There you can mount the camera to continuously monitor the packs for quality control. The source code for the system runs in an infinite loop. But for actual implementation, you need to adapt it and make it trigger based, so that processing happens only when the blister pack arrives under the camera.

The camera captures an image of the pack and converts it into one that shows hue difference from your predefined value. This means that pills will be shown as black because only at their places there will be no difference in hue from the predefined value. This image is inverted, so pills appear white and the rest of the surrounding area turns black.

The objective is to pre-process the image so that the pills area appears as blobs to get detected. The blobs are easy to count and the blob count indicates the number of pills in the package. This number is compared with the desired number of pills for each package, thus miss-

ing pills are automatically detected. The software beeps if the number of detected pills matches the desired value. You can also program it to do the opposite.

The steps to run the software are reproduced below for better understanding:

Run Python IDLE→File→New File

Paste the program or use the file Quality Control.py.

Enter right values for:

1. Camera (number?)
2. peakHue (that you have found using the second program hue.py)
3. Threshold through hit-and-trial method
4. minszie for blobs detection. This depends on the placement of your camera. Use hit-and-trial method to get the correct value
5. le == ?? (put the number of pills desired in your packs)

Now click Run followed by Run Module.

You will immediately see a screen with various processed images such as the one shown in Fig. 1. If the number of detected pills is equal to the desired value, the software will produce a beep sound and show OK on the console. Otherwise, it will show Fail on the console. **EFY**

EFY Note

The source code of this project and Appendices 1 and 2 are included in this month's EFY DVD and are also available for free download at source.efymag.com



Pooja Juyal is manager at Samtel Avionics Ltd